Introduction to Database Systems and ACID Properties

A relational database organizes data into tables of rows and columns and is usually queried using SQL. Transactions group multiple operations so they behave as a single unit of work, and relational databases guarantee the ACID properties for transactions: Atomicity, Consistency, Isolation, and Durability.

Atomicity means a transaction either fully completes or has no effect at all. Consistency means a transaction moves the database from one valid state to another, respecting all constraints. Isolation means concurrent transactions do not see each other's uncommitted intermediate states. Durability means that once a transaction commits, its effects survive even a subsequent crash.

Normalization: 1NF, 2NF, and 3NF

Normalization is the process of structuring tables to reduce redundancy and avoid update anomalies. First Normal Form (1NF) requires that every column hold a single atomic value and that rows have no repeating groups.

Second Normal Form (2NF) requires 1NF plus that every non-key column depends on the entire primary key, not just part of a composite key. Third Normal Form (3NF) requires 2NF plus that every non-key column depends only on the primary key and not on another non-key column (eliminating transitive dependencies).

Indexing and B-Trees

An index is an auxiliary data structure that lets a database find rows matching a condition without scanning the entire table. Most relational databases implement indexes using a B-tree or one of its variants, such as the B+ tree.

A B-tree keeps its keys sorted and balanced so that lookups, insertions, and deletions all take logarithmic time relative to the number of rows. B+ trees store all actual data (or row pointers) in the leaf nodes and link the leaves together, which makes range scans over sorted keys efficient.

SQL Joins

A join combines rows from two or more tables based on a related column. An INNER JOIN returns only the rows that have matching values in both tables. A LEFT (OUTER) JOIN returns all rows from the left table, with NULLs filling in columns from the right table when there is no match.

A RIGHT JOIN is the mirror image of a LEFT JOIN, keeping all rows from the right table. A FULL OUTER JOIN keeps all rows from both tables, filling in NULLs on whichever side lacks a match.

Transactions and Concurrency Control

When multiple transactions run concurrently, a database must prevent problems such as dirty reads (reading another transaction's uncommitted data), lost updates, and non-repeatable reads. Isolation levels, such as Read Committed, Repeatable Read, and Serializable, trade off strictness against concurrency.

Two common concurrency-control techniques are locking, where a transaction acquires locks on the rows or tables it touches before releasing them at commit, and multiversion concurrency control (MVCC), where readers see a consistent snapshot of the data without blocking writers.

CAP Theorem and NoSQL Databases

The CAP theorem states that a distributed data store can provide at most two of three guarantees at the same time during a network partition: Consistency (every read sees the latest write), Availability (every request gets a response), and Partition tolerance (the system keeps working despite network partitions).

NoSQL databases, such as document stores, key-value stores, and wide-column stores, often relax strict consistency in favor of availability and horizontal scalability, compared to traditional single-node relational databases.